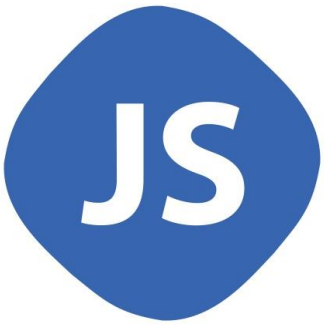


Атрибуты и стили 2

- Методы для работы с атрибутами
- Синхронизация свойств и атрибутов
- Нестандартные атрибуты
- Свойства и атрибуты. Вопросы

Методы для работы с атрибутами

Помимо свойства **attributes**, которое возвращает атрибуты в виде **NameNodeMap**, у каждого элемента есть методы, которые позволяют работать с его атрибутами **более просто**:

- **getAttribute**()
- **setAttribute**()
- **removeAttribute**()
- **hasAttribute**()

Кроме того, методы позволяют получить **доступ к нестандартным атрибутам**, для которых не будет соответствующих DOM-свойств.

Эти методы работают именно с HTML-атрибутами как они есть в документе. Фактически, они изменяют код страницы. Поэтому для них справедливо:

- Значения атрибутов - это **всегда строки**.
- Имя атрибута можно указывать в любом регистре, так как HTML не предъявляет требований к регистру символов в коде.

getAttribute

Метод **getAttribute**() возвращает значение атрибута, указанного в качестве параметра. Если данного атрибута у элемента нет, то данный метод возвращает **пустую строку** или **null**.

Синтаксис:



```
node.getAttribute (nameAttr); // получить атрибут
```

Параметры:

- **nameAttr** - имя атрибута.

В следующем примере получаем значения **стандартных** атрибутов узла, соответствующего HTML-тегу div.

example_1. Метод getAttribute

```
<script>
	let header = document.body.firstElementChild;
	let div = header.nextElementSibling;
	// методы доступа к атрибутам
	console.log("id = ", div.getAttribute ("id"));
	console.log("class = ", div.getAttribute ("class"));
</script>
```

Вы прочитали фразу - "Значения атрибутов - это всегда строки" и пошли дальше. Или не было такой фразы?). Смотрим пример.

example_2. Значение атрибута всегда строка

```
script>
	// ссылка на узел div
	let div = document.body.firstElementChild.nextElementSibling;
	// свойство style
	console.log (div.style); // объект
	// атрибут style
	console.log (div.getAttribute('style')); // строка
</script>
```



С помощью свойства **attributes** или специальных **методов __Attribute**() можно получить доступ к атрибутам вне зависимости от того, **стандартный он для выбранного элемента или нет**.

example_3. Стандартные и нестандартные атрибуты

```
<script>
	let header = document.body.firstElementChild;
	let div = header.nextElementSibling;
	// свойства из атрибутов
	console.log("id = ", div.id);
	console.log("class = ", div.className);
	console.log("title = ", div.title);
	// свойство из нестандартного атрибута
	console.log("name = ", div.name); // undefined
	console.log("----------");
	// свойства и методы доступа к атрибутам
	console.log("id = ", div.getAttribute ("id"));
	console.log("class = ", div.attributes.item(1).value);
	console.log("title = ", div.attributes.getNamedItem ("title"));
	// доступ к нестандартному атрибуту
	console.log("name = ", div.getAttribute ("name"));
</script>
```

Пользовательские атрибуты, разработчик **придумывает свои атрибуты** для тега. Делать так не совсем хорошо. Если есть необходимость использовать такой атрибут, для таких задач есть цивилизованный способ (смотреть далее).

example_4. Пользовательские атрибуты

```
<script>
	let header = document.body.firstElementChild;
```

```
    let div = header.nextElementSibling;
    // нестандартный атрибут
    console.log ("Нестандартный атрибут name: ");
    console.log("name = ", div.getAttribute ("name"));
    console.log(div.attributes.getNamedItem ("name"));
    // придуманные атрибуты
    console.log ('Придуманные атрибуты: ');
    console.log("book = ", div.getAttribute ("book"));
    console.log(div.attributes.getNamedItem ("author"));
</script>
```

Важно. Стандартные атрибуты **описаны в спецификации для соответствующего класса элемента** (легко найти в открытых источниках).

setAttribute

Метод **setAttribute**() добавляет указанный атрибут к элементу и присваивает ему указанное значение. Если указанный атрибут у элемента уже есть, то данный метод изменяет только его значение.

Синтаксис:

```
node.setAttribute (nameAttr, value); // установить атрибут
```

Параметры:

- **nameAttr** - имя атрибута.
- **value** - значение атрибута.

Вспомним, как мы назначали атрибут в прошлом уроке.

example_5. Создание узла атрибута

```
<script>
	// ссылка на узел div
	let div = document.body.firstElementChild.nextElementSibling;
	// создаем атрибут
	var attr = document.createAttribute("id");
	// назначим атрибуту значение
	// к узлу применились настройки из <style>
	attr.value = "citate";
	div.attributes.setNamedItem(attr);
</script>
```

... и по достоинству оценим метод **setAttribute**().

example_6. Метод setAttribute

```
<script>
	// ссылка на узел div
	let div = document.body.firstElementChild.nextElementSibling;
	// установим атрибут id
	// к узлу применились настройки из <style>
	div.setAttribute ("id", "citate");
	console.log("Текущий идентификатор элемента: id =",
	div.getAttribute("id"));
</script>
```

removeAttribute

Метод **removeAttribute**() удаляет указанный атрибут у элемента. Данный метод имеет один обязательный параметр - это строка, содержащая имя атрибута, который вы хотите удалить у элемента. Данный метод в качестве результата ничего не возвращает.



Синтаксис:

```
node.removeAttribute (nameAttr); // удалить атрибут
```

Параметр:

- **nameAttr** - имя атрибута.

example_7. Метод RemoveAttribute

```
<script>
	// ссылка на узел div
	let div = document.body.firstElementChild.nextElementSibling;
	// вывод списка узлов атрибутов
	console.log (div.attributes);
	// удаляем атрибуты
	div.removeAttribute ("book");
	div.removeAttribute ("author");
	// вывод списка узлов атрибутов
	console.log (div.attributes);
</script>
```

hasAttribute

Метод **hasAtrribute**() возвращает **true**, если указанный атрибут существует у элемента. В противном случае данный метод возвращает **false**. Данный метод имеет один обязательный параметр - это строка, содержащая имя атрибута, который вы хотите проверить на существование у элемента.

Синтаксис:

```
node.hasAttribute(nameAttr); // проверить наличие атрибута
```

Параметр:

- **nameAttr** - имя атрибута.

example_8. Метод hasAttribute

```
<script>
	// ссылка на узел body
	let body = document.body;
	let header = body.firstElementChild;
	let div = header.nextElementSibling;
	// true - если атрибут присутствует
	console.log (div.hasAttribute ("id"));
</script>
```

Синхронизация свойств и атрибутов

В предыдущем уроке я говорил, в DOM-интерфейсе понятия HTML-атрибутов и DOM-свойств элементов **тесно связаны**. При создании модели страницы для каждого элемента создаются одноимённые с атрибутами свойства и присваиваются им те же значения. Однако, это справедливо не для всех атрибутов.

Когда **стандартный** атрибут изменяется, соответствующее свойство автоматически обновляется.

В следующем примере id **модифицируется как атрибут**, и можно увидеть, что **свойство** также **изменено**.

example_9. Синхронизируем атрибут -> свойство

```
<script>
	// ссылка на узел div
	let div = document.body.firstElementChild.nextElementSibling;
	// меняем атрибут
	div.setAttribute('id', 'myNewId');
```

```
    console.log("Атрибут: ", div.getAttribute('id'));
    // меняется свойство
    console.log ("Свойство: ", div.id);
</script>
```

То же самое работает и в обратную сторону, **меняем свойство**, **меняется атрибут**.

example_10. Синхронизируем свойство -> атрибут

```
<script>
    // ссылка на узел div
    let div = document.body.firstElementChild.nextElementSibling;
    // меняем свойство
    div.id = "myNewId";
    console.log ("Свойство: ", div.id);
    // меняется атрибут
    console.log ("Атрибут: ", div.getAttribute('id'));
</script>
```

Изменение свойства обновило атрибут. Но это скорее **исключение**, чем правило, синхронизация не гарантирует одинакового значения в атрибуте и свойстве.

> ***К сведению***. Все **стандартные** свойства DOM синхронизируются с атрибутами, однако не всегда такая синхронизация происходит один-в-один. Чаще синхронизация – **односторонняя**: **меняется атрибут - меняется свойство**, но не наоборот.

Рассмотрим пример, когда при изменении **свойства** "value" **атрибут** "value" не меняется.

example_11. Отсутствие синхронизации в свойстве value

```
<script>
	// ссылка на узел input
	let input = document.body.firstElementChild.nextElementSibling;
	// изменим свойство
	input.value = 'Shape of My Heart';
	// атрибут не поменялся
	console.log (input.getAttribute('value') );
</script>
```

Или другой пример, что произойдёт с атрибутом "href" при изменении свойства:

example_12. Отсутствие синхронизации в свойстве href

```
<script>
	// ссылка на узел a
	let a = document.body.firstElementChild.nextElementSibling;
	// свойство href
	console.log(a.href);
	// атрибут href
	console.log (a.getAttribute('href'));
	// изменим свойство
	a.href = "./";
	// свойство href
	console.log(a.href);
	// атрибут href
	console.log (a.getAttribute('href'));
</script>
```

Это происходит потому, что **атрибут может быть любым**, а свойство **href**, в соответствии со спецификацией W3C, должно быть **полной ссылкой**.

Таким образом. Если мы хотим получить именно то, что хранится в HTML, нужно обращаться через **атрибут**.

Для нестандартных и придуманных атрибутов **синхронизация не работает ни в одну из сторон**.

example_13. Отсутствие синхронизации в нестандартных/пользовательских атрибутах

```
<script>
	// ссылка на узел div
	let div = document.body.firstElementChild.nextElementSibling;
	// меняем нестандартный атрибут
	div.setAttribute('name', 'Похождения Чичикова, или Мертвые души');
	// меняем придуманный (пользовательский атрибут)
	div.setAttribute('myAttr', 'Дата выхода: 1835')
	console.log (div.name);  // undefined
	console.log (div.myAttr);  // undefined
	// через метод проверяем новое значение
	console.log(div.getAttribute("name"));
</script>
```

Совет. Если вы начинающий разработчик не пытайтесь запомнить все и сразу. Оттачивайте мастерство не только на моих примерах, придумывайте свои. Возвращайтесь к материалу, если чувствуете что забыли. Второй раз достаточно будет просмотреть по диагонали текст и вы вспомните о чем там речь. И каждый раз в знакомых строчках вы будете находить новые смыслы, это нормально.



Нестандартные атрибуты

При написании HTML мы используем много стандартных атрибутов. Но что насчёт нестандартных, пользовательских?

Нестандартные атрибуты элементов могут использоваться:

- для передачи пользовательских данных из **HTML** в **JavaScript**;
- для обозначения (маркирования) HTML-элементов для JavaScript.

Прежде чем продолжить, еще раз о главном.

Нужно хорошо понимать, что значит понятие **нестандартный атрибут**. Атрибут **name** стандартный для тега **input**, но нестандартный для тега **div**. Придуманный мной атрибут **myAttr** – нестандартен в принципе.

С пользовательскими (придуманными) атрибутами могут возникнуть проблемы. Что если мы используем нестандартный атрибут для наших целей, а позже он **появится в стандарте**, и будет выполнять какую-то функцию? Язык HTML живой, он растёт, появляется больше атрибутов, чтобы удовлетворить потребности разработчиков. В таком случае могут возникнуть неожиданные и нежелательные эффекты.

Чтобы избежать потенциально возможных конфликтов, существуют атрибуты вида **data-***.

> ***Важно***. Все атрибуты, начинающиеся с префикса **data-**, **зарезервированы** для использования программистами. Атрибуты **data** доступны в свойстве **dataset**.

Например, если у elem есть атрибут "data-about", то обратиться к нему можно как elem.dataset.about.

example_14. Свойство узла dataset

```
<script>
```



```
        // ссылка на узел div
        let div = document.body.firstElementChild.nextElementSibling;
        // доступ через метод к нестандартным атрибутам
        console.log(div.getAttribute ("data-book"));
        console.log(div.getAttribute ("data-author"));
        console.log("----------")
        // свойства из нестандартных атрибутов
        console.log(div.dataset.book);
        console.log(div.dataset.author);
</script>
```

Примечание. Если имя атрибута **data**- содержит тире, например так: data-my-login, то свойство dataset будет писаться в нотации camelCase: **dataset.myLogin**.

Свойства и атрибуты. Вопросы

Каждая написанная строка вроде бы вполне себе понятна. Тем более, когда строка подкреплена конкретным примером. Но вдумчивый читатель может задать себе некоторые вопросы, и не найти на них ответа. Имея опыт работы со студентами, попробую предугадать эти вопросы.

В начале и на протяжении всей темы говорилось, что браузер преобразует HTML-код документа в DOM-модель и работает с узлами этой модели. Но в последних уроках неоднократно говорилось про HTML-атрибуты и DOM-свойства.

Получается, браузер работает с DOM моделью документа, но нет-нет, да и заглянет в HTML-код для поиска интересующих нас атрибутов?



Смотрите.

Это ссылка на элемент **div**:

```
let div = document.body.firstElementChild.nextElementSibling;
```

Элемент div - это **HTML-узел** в дереве DOM документа. Узел DOM – это javaScript объект, который содержит очень много **свойств** и методов.

```
// узел div
console.dir (div);
```

В числе прочих, узел div имеет свойство **attributes**.

Свойство **attributes** - возвращает **группу атрибутов, зарегистрированных в указанном узле**. Каждый **атрибут это строка** (пара ключ/значение), которая представляет собой информацию относительно этого атрибута.

```
// attributes - одно из свойств узла
console.dir (div.attributes);
```

Таким образом, узел div:

- это сложный объект, который имеет набор **встроенных** и **пользовательских** свойств, а так же свойств, полученных из **стандартных** атрибутов.
- имеет свойство **attributes**, которое хранит всю информацию об атрибутах тега в **строках**.

Можно посмотреть **свойство** "**title**" узла div:

```
// просмотр свойства title
console.log (div.title);
```

Можно посмотреть **атрибут** "**title**" узла div:

```
// просмотр атрибута title
console.log (div.attributes.getNamedItem('title'));
```

С атрибутами удобнее работать с помощью **специальных методов**:

```
// просмотр атрибута title в более удобной форме
console.log (div.getAttribute('title'));
```

Элементы из набора **attributes** синхронизируются со **свойствами** узла div (раздел урока: Синхронизация свойств и атрибутов).

example_15. Свойства и атрибуты

```
<script>
	// ссылка на узел div
	let div = document.body.firstElementChild.nextElementSibling;
	// узел div
	console.dir (div);
	// attributes - одно из свойств узла
	console.dir (div.attributes);
	// просмотр свойства title
	console.log (div.title);
	// просмотр атрибута title
	console.log (div.attributes.getNamedItem('title'));
	// просмотр атрибута title в более удобной форме
	console.log (div.getAttribute('title'));
</script>
```

Если совсем просто, то:

- **Атрибуты** - это то, что написано в HTML и хранится в attributes.
- **Свойство** - это то, что хранится в DOM-объекта.

Так каким все-таки образом связаны свойства и атрибуты?

Синхронизация между атрибутами и свойствами:



Стандартные свойства и атрибуты синхронизируются: **установка атрибута автоматически ставит свойство DOM**. Некоторые свойства синхронизируются в обе стороны.

Связь свойств с атрибутами:

Бывает так, что имя свойства не соответствует имени атрибута.

Нестандартные атрибуты:

Нестандартный атрибут никогда не попадёт в свойство, так что для кросс-браузерного доступа к нему нужно обязательно использовать getAttribute.

Когда же использовать свойства, а когда атрибуты?

В большинстве ситуаций предпочтительнее использовать **DOM-свойства**, а атрибуты только тогда, когда нужны именно атрибуты, например:

- нужен нестандартный атрибут (но если он начинается с data-, тогда нужно использовать dataset);
- необходимо получить оригинальное значение, которое написано в HTML (например, свойство href – всегда полный URL, а значение атрибута может отличаться).

Если вы хотите использовать собственные атрибуты в HTML, помните, что атрибуты с именем, начинающимся на **data**- валидны в HTML5 и современные браузеры поддерживают доступ к ним через свойство **dataset**.

Самостоятельные задания к уроку:

- Назначение узлу атрибутов.
- Назначение узлу свойств.
- Операции с узлами и атрибутами.
- Объекты javaScript, свойства объекта в роли атрибутов узла.

